

7 - UML

Modello

Un modello viene definito come una rappresentazione semplificata della realtà

La creazione di modelli viene resa necessaria a causa della complessità del mondo reale, in quanto tali astrazioni facilitano significativamente la comprensione dei sistemi e favoriscono la comunicazione tra le varie parti interessate

Punti di vista sulla modellazione

Esistono due punti di vista nella modellazione:

- **Prospettiva concettuale:** si concentra sulla rappresentazione dei concetti caratteristici del dominio oggetto di studio, dando vita a quello che viene definito "domain model", il quale risulta essere totalmente indipendente dalle logiche di implementazione del software
- **Prospettiva software:** gli elementi presenti in un modello corrispondono in modo diretto agli elementi del sistema informatico

Questo approccio può essere adottato sia in una fase precedente allo sviluppo effettivo del sistema, processo noto come **forward engineering**, sia partendo dall'analisi di un sistema già esistente, pratica definita **reverse engineering**

Linguaggio UML

L'acronimo UML sta per **Unified Modeling Language** e rappresenta una famiglia di notazioni grafiche standardizzate destinate alla modellazione visuale dei sistemi software. Questo linguaggio trova il suo impiego primario nelle fasi di analisi e progettazione orientata agli oggetti.

A seconda delle esigenze del progetto, l'UML può essere impiegato con tre differenti livelli di rappresentazione:

- Semplice abbozzo
- Design tecnico dettagliato
- Linguaggio di programmazione

UML come abbozzo

L'utilizzo dell'UML come abbozzo, o "sketch", ha lo scopo principale di stimolare la comprensione e la comunicazione durante le discussioni di team, permettendo di focalizzare l'attenzione solo su determinati aspetti di un sistema software.

I criteri fondanti di questo approccio sono la **selettività** e l'**espressività**:

- La selettività implica che solo alcuni dettagli vengano modellati, potendo omettere liberamente informazioni non essenziali
- L'espressività è la creazione di diagrammi intesi come figure collaborative, spesso abbozzate improvvisando su supporti informali come le lavagne

UML come design tecnico

Lo scopo principale di usare l'UML come design tecnico dettagliato (chiamato anche **blueprint**) è quella di dare uno schema da seguire ai programmatori.

Il linguaggio quindi serve a guidare e documentare fedelmente la realizzazione del software, dovendo pertanto rispettare criteri di completezza e non ambiguità

I diagrammi creati diventano parte integrante e formale della documentazione del sistema

UML come linguaggio di programmazione

L'UML come linguaggio di programmazione mira a consentire agli sviluppatori di programmare in modalità visuale, mantenendo una totale indipendenza dalla piattaforma software .

I diagrammi vengono compilati in modo diretto per generare formati eseguibili, un paradigma che prende il nome di Model Driven Development (MDD),

Diagrammi UML

L'UML si divide in due macro categorie di diagrammi:

- **Diagrammi strutturali**
- **Diagrammi comportamentali**

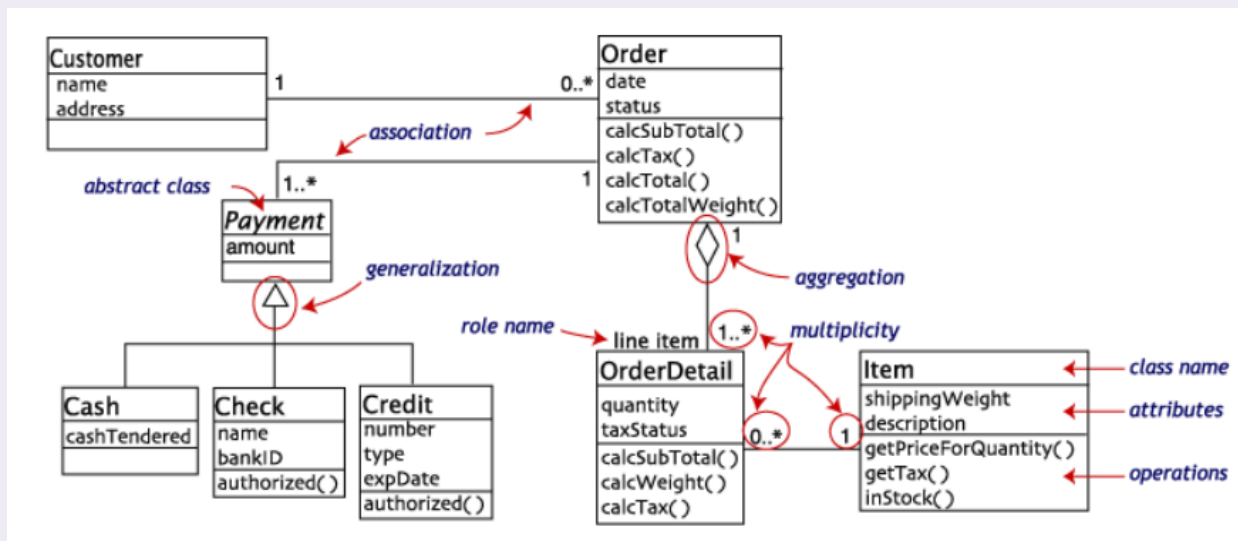
Diagrammi strutturali

I diagrammi strutturali descrivono l'architettura statica del sistema e includono i seguenti diagrammi che andremo a vedere

Diagramma delle classi

Il diagramma delle classi mostra le classi, le loro caratteristiche interne e le reciproche relazioni

Esempio di diagramma delle classi



È uno strumento largamente utilizzato sia nella modellazione concettuale che in quella software

Diagramma dei componenti

Il diagramma dei componenti illustra i moduli del sistema e le relative connessioni

≡ Esempio di diagramma dei componenti

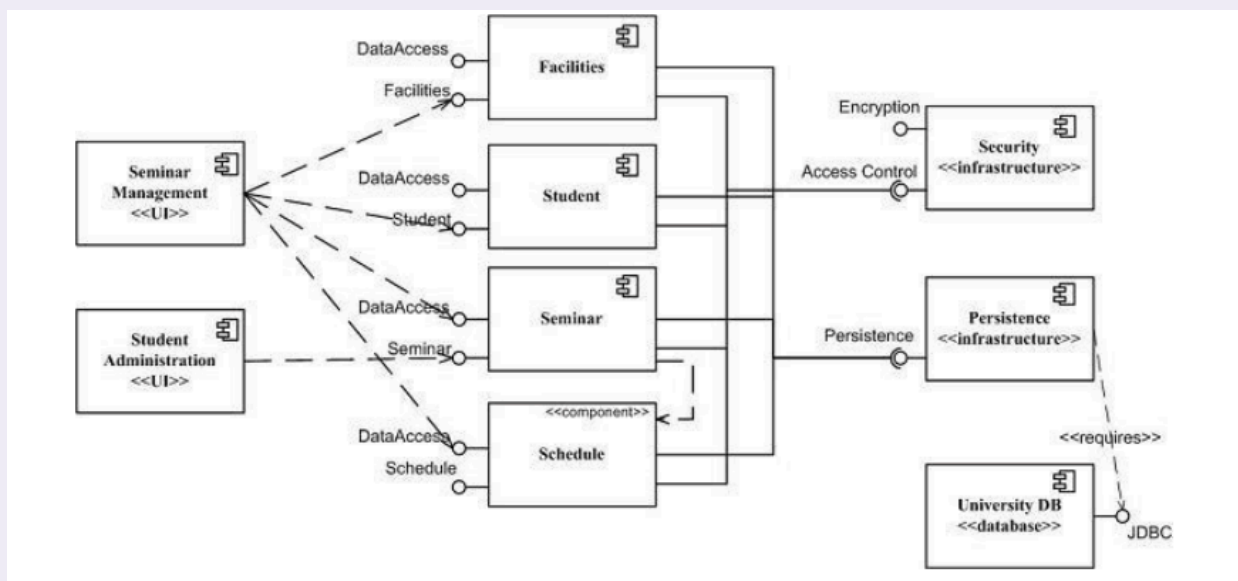
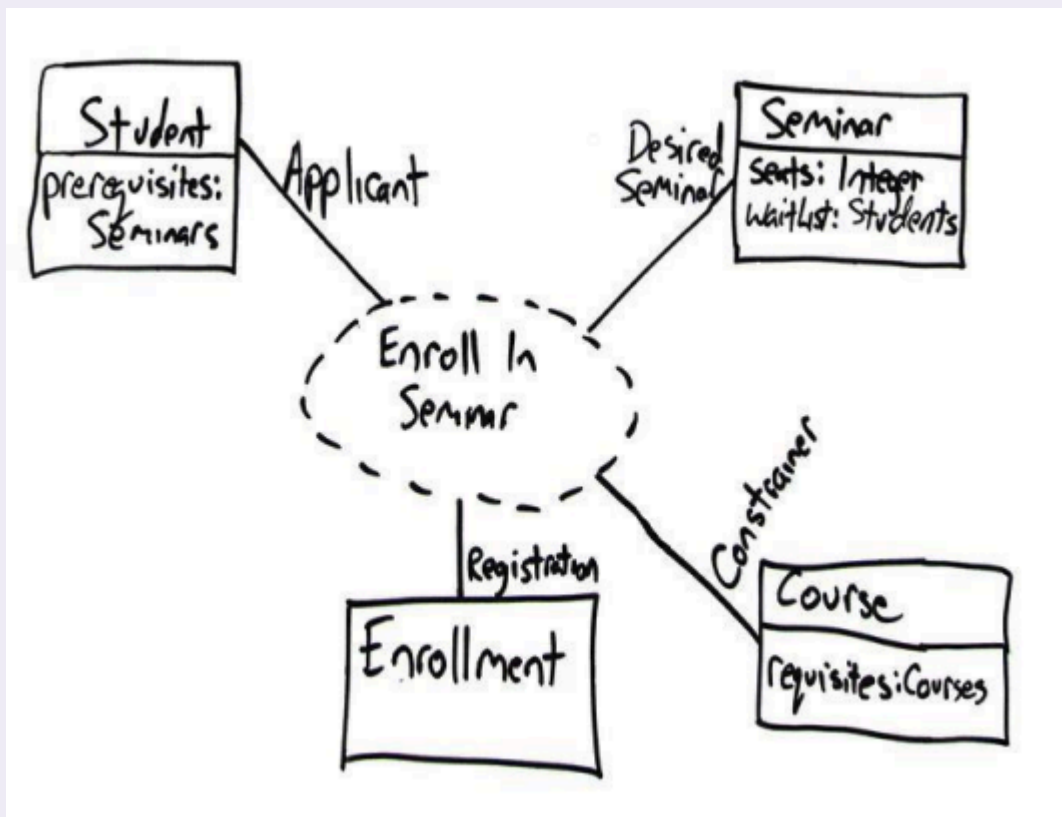


Diagramma di struttura composita

Il diagramma di struttura composita, introdotto a partire dalla versione UML 2, mostra in dettaglio la scomposizione a runtime di un classificatore

≡ Esempio di diagramma di struttura composita



Utile per esplicitare le realizzazioni dei casi d'uso

Diagramma di deployment

Il diagramma di deployment mostra la mappa la distribuzione fisica dei componenti software nei diversi nodi hardware di elaborazione

≡ Esempio di diagramma di deployment

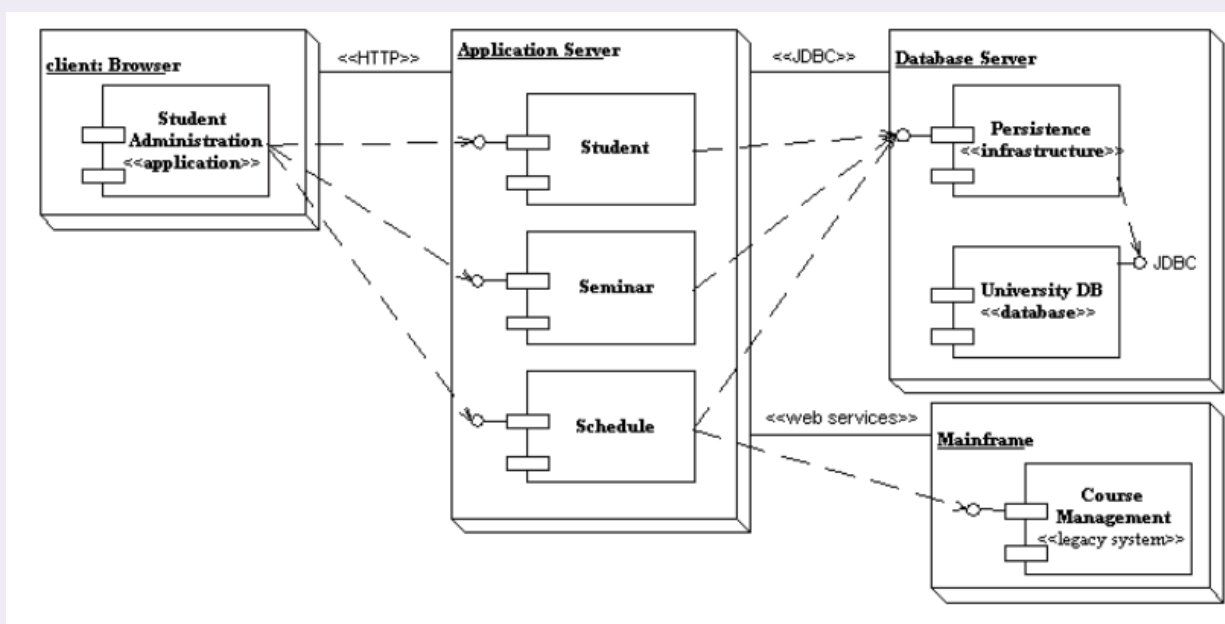


Diagramma degli oggetti

Il diagramma degli oggetti fornisce un'istantanea di una configurazione di istanze e dei loro collegamenti in un preciso momento temporale

≡ Esempio di diagramma degli oggetti

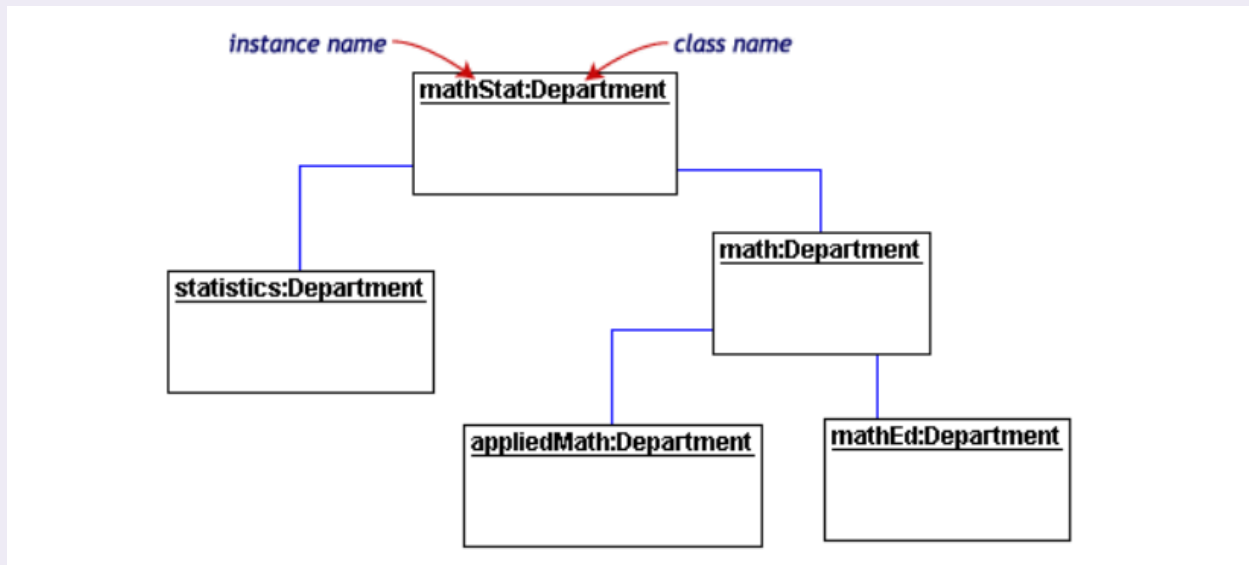


Diagramma dei package

Il diagramma dei package visualizza le dipendenze tra costrutti di raggruppamento (i package, appunto, che possono contenere classi, casi d'uso o componenti). Si rivela indispensabile nei progetti di grande scala.

≡ Esempio di diagramma dei package

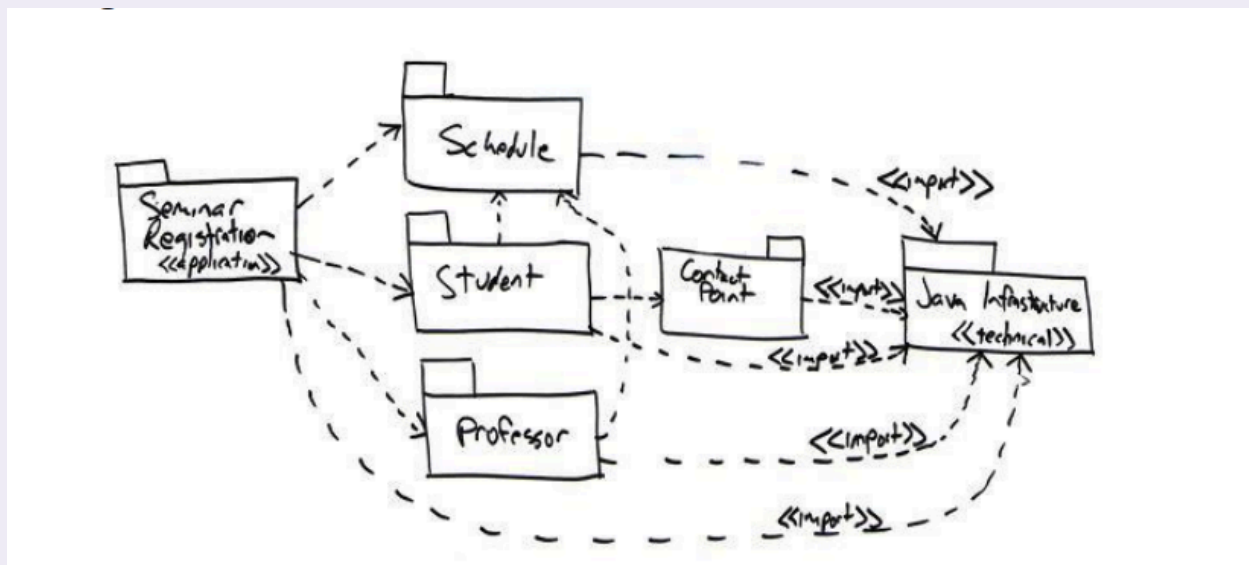


Diagramma comportamentale

I diagrammi comportamentali modellano la dinamica del sistema

Diagramma delle attività

Il diagramma delle attività è atto a rappresentare la modellazione dei processi e dei workflow, descrivendo comportamenti sia procedurali che paralleli

≡ Esempio di diagramma delle attività

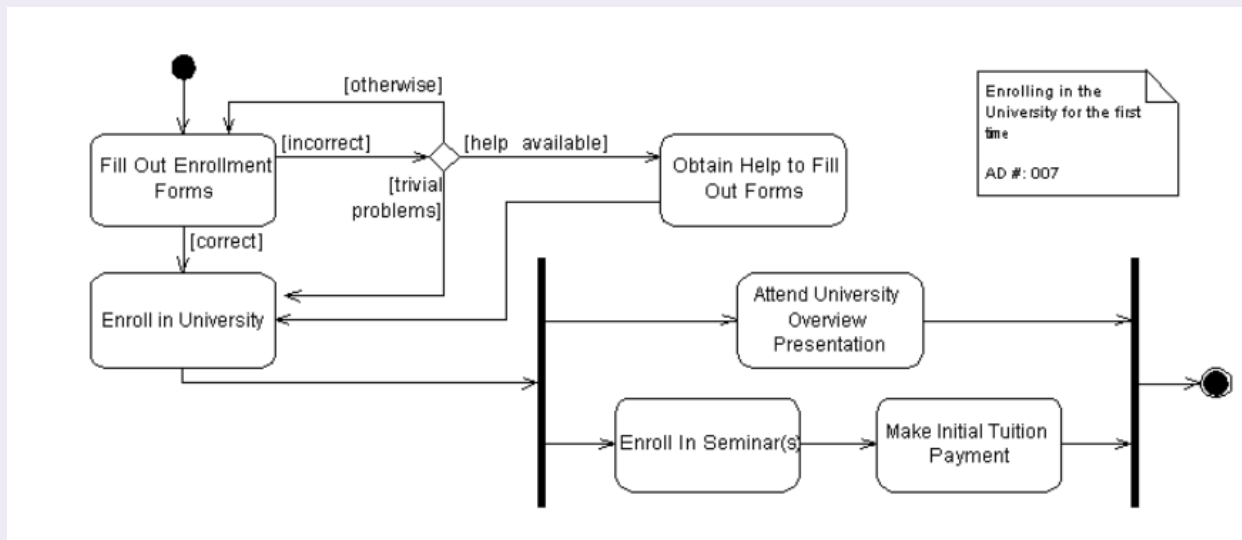


Diagramma dei casi d'uso

Il diagramma dei casi d'uso espone il modo in cui gli attori o gli utenti interagiscono con il sistema, sebbene la rappresentazione grafica in sé sia considerata meno rilevante rispetto alle descrizioni testuali associate

≡ Esempio di diagramma dei casi d'uso

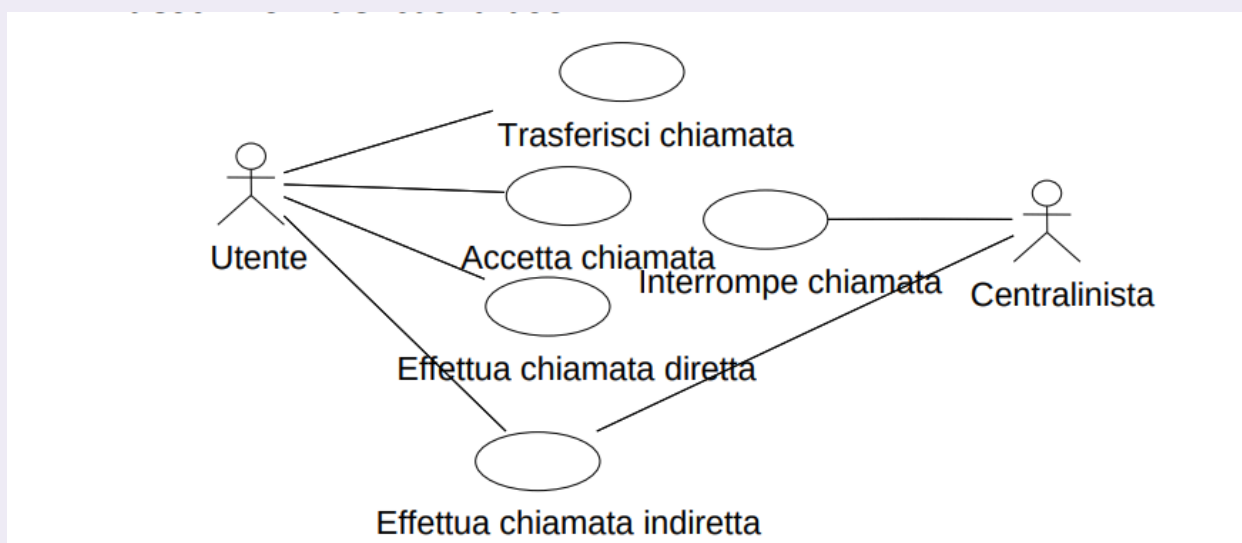


Diagramma di macchina a stati

Il diagramma di macchina a stati evidenzia come eventi specifici modifichino un oggetto lungo il suo ciclo di vita

≡ Esempio di diagramma di macchina a stati

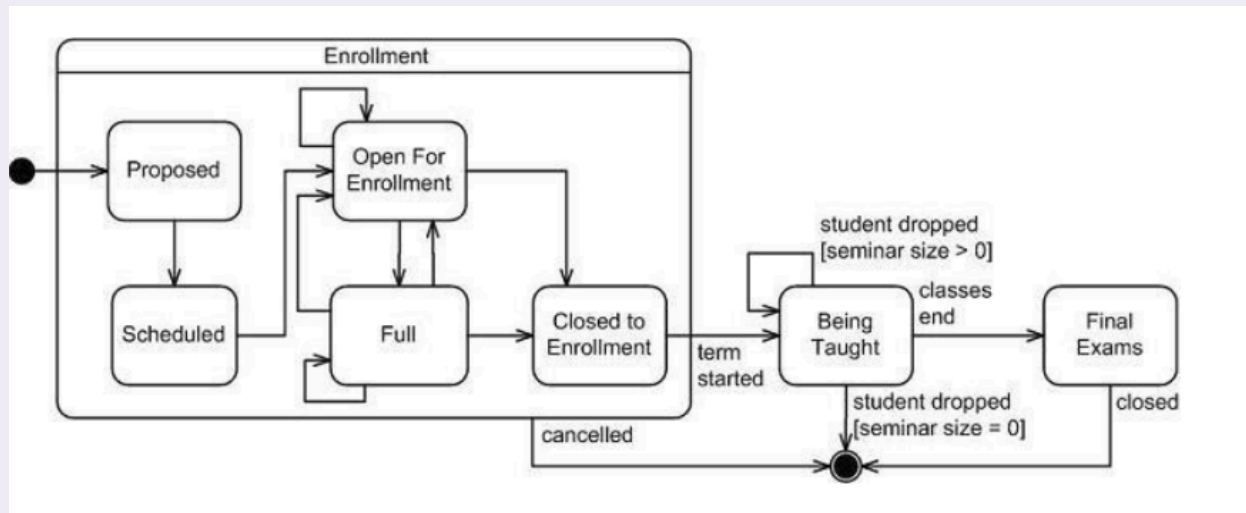


Diagramma di sequenza

Il diagramma di sequenza mostra l'ordine cronologico dei messaggi scambiati tra gli oggetti ed è massicciamente impiegato per illustrare il comportamento in specifici scenari funzionali

≡ Esempio di diagramma di sequenza

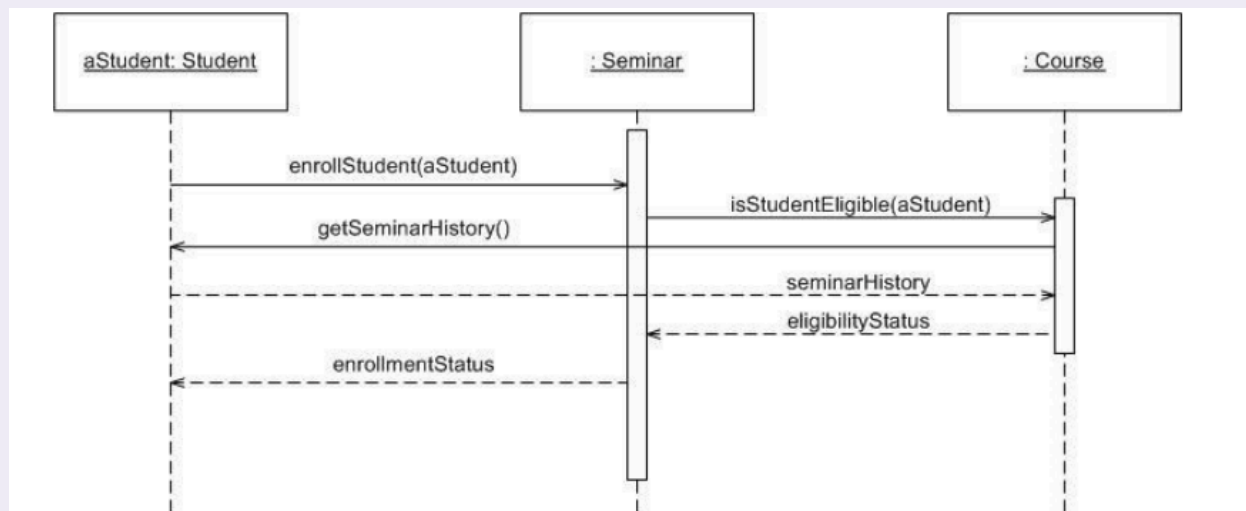


Diagramma di comunicazione

Il diagramma di comunicazione è logicamente equivalente a quello di sequenza, ma sposta l'enfasi visiva sui collegamenti strutturali tra le entità, risultando tuttavia meno utilizzato

≡ Esempio di diagramma di comunicazione

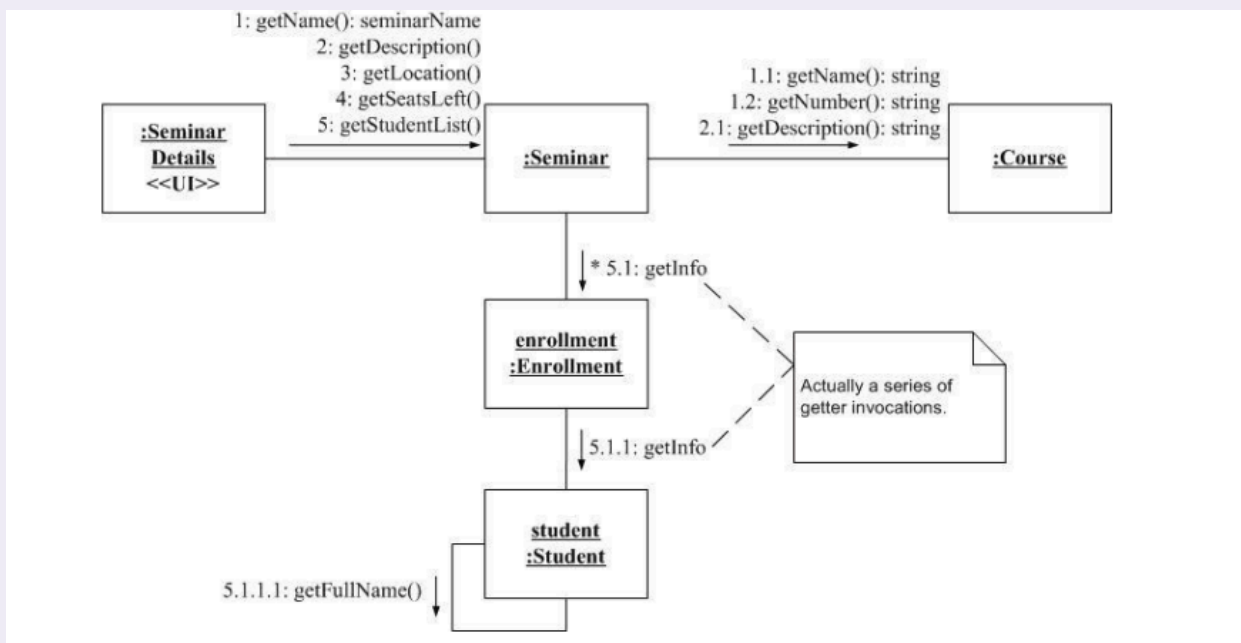


Diagramma di interazione generale

Il diagramma di interazione generale, novità di UML 2, costituisce una fusione concettuale tra un diagramma di sequenza e uno delle attività

≡ Esempio di diagramma di interazione generale

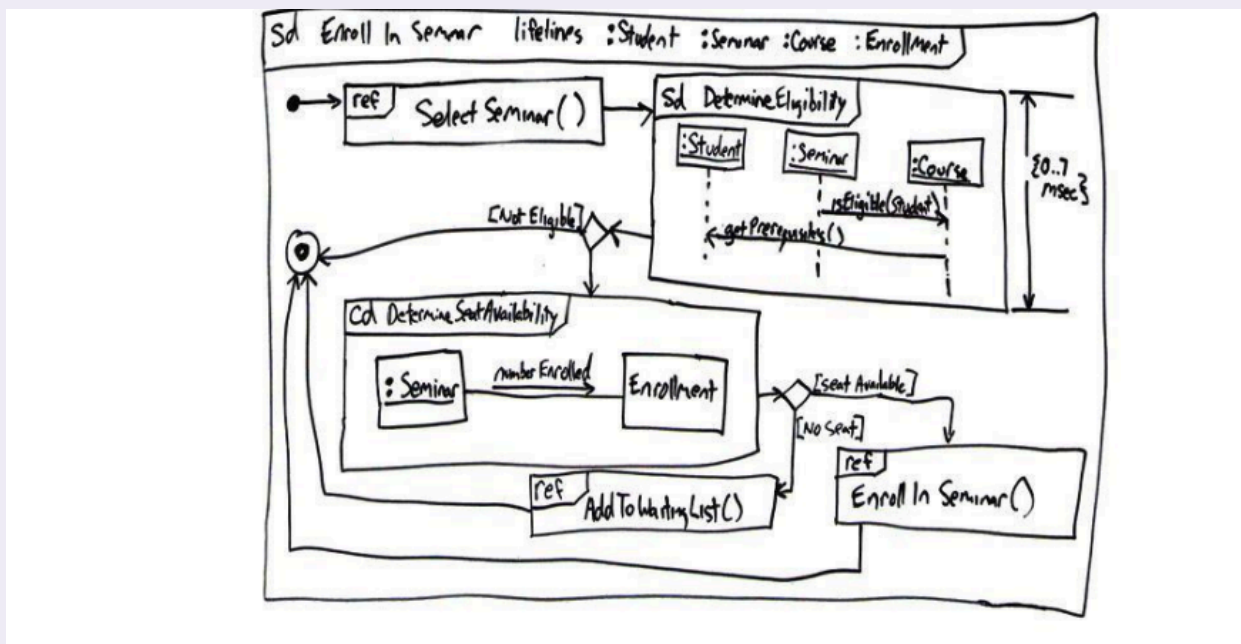
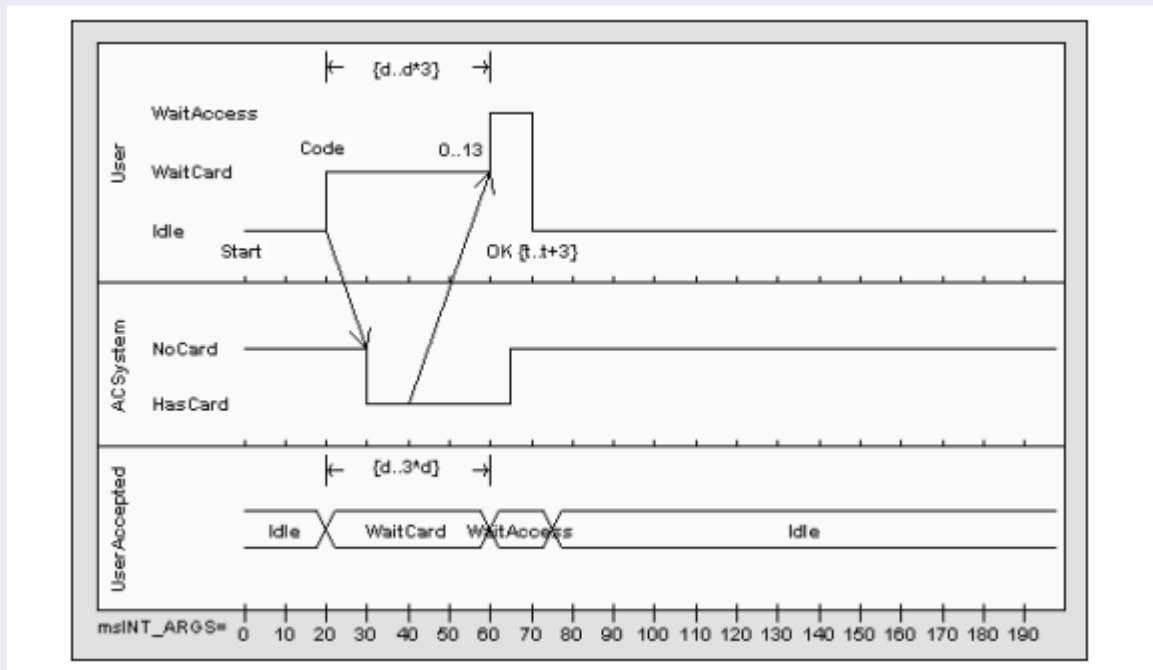


Diagramma di temporizzazione

Il diagramma di temporizzazione, anch'esso introdotto con UML 2, si concentra rigorosamente sui vincoli temporali ed è adottato specialmente nell'ambito dei software

embedded e real-time, oltre a essere uno strumento d'elezione per gli ingegneri impegnati nella progettazione elettronica

≡ Esempio di diagramma di temporizzazione



Utilizzo dei diagrammi UML

Generalmente, in diverse fasi di progettazione, vengono utilizzati i seguenti diagrammi UML:

- **Analisi dei requisiti:** diagramma delle classi, diagramma delle attività, diagramma di macchina a stati
- **Attività di design:** diagramma delle classi, diagrammi di sequenza